Chapter 11. Generative Models and Autoencoders

Lecture 11/9: supervised vs unsupervised learning, generative modeling, PixelRNN and PixelCNN, latent variable models, autoencoders, autoencoders for generation, the variational lower bound

Contents

1 Supervised versus Unsupervised Learning **2**

2 Generative Modeling **2**

3 Autoregressive Models **2**

4 Latent Variable Models **3**

5 Autoencoders **4**

6 Variational Autoencoder **4**

1 Supervised versus Unsupervised Learning

	supervised	unsupervised
data	pairs (x, y), y a label	x only, no labels
goal	learn a map $x \mapsto y$	learn hidden structure of the data
examples	classification, regression, detection, segmentation, captioning	clustering, dimensionality reduction (PCA, 3-d $\to$ 2-d), density estimation

2 Generative Modeling

Motivation. Given training data, generate new samples from the same distribution.

Training data $\sim p_{\text{data}}(x)$; learning gives $p_{\text{model}}(x)$, sampling gives new x. Objectives:

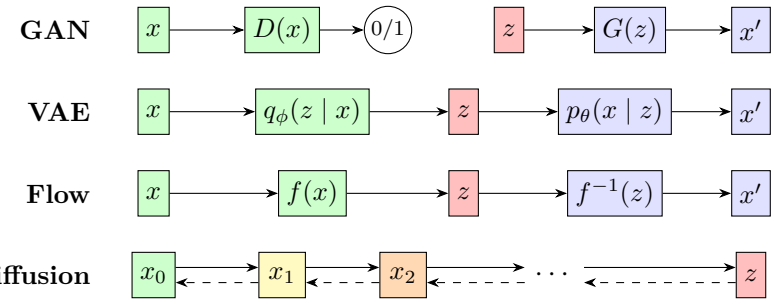
1. learn $p_{\text{model}}(x)$ approximating $p_{\text{data}}(x)$;
2. sample new x from $p_{\text{model}}(x)$.

As density estimation:

- **Explicit**: define and solve for $p_{\text{model}}(x)$ explicitly.
- **Implicit**: learn a model that samples from $p_{\text{model}}(x)$ without defining it.

Four families. x: training data, x': generated sample, z: latent variable with a simple, tractable distribution.

- **GAN**: adversarial training; generator $G(z) = x'$, discriminator $D(x) \in \{0, 1\}$.
- **VAE**: maximize a variational lower bound; encoder $q_\phi(z \mid x)$, decoder $p_\theta(x \mid z)$.
- **Flow-based**: invertible transform $z = f(x)$, $x' = f^{-1}(z)$.
- **Diffusion**: gradually add Gaussian noise $x_0 \to x_1 \to \cdots \to z$, then reverse.



Green: encoding / real data; blue: generation; red: latent variable with a simple distribution.

3 Autoregressive Models

An RNN models $p(x_i \mid x_1, \ldots, x_{i-1})$ (Chapter 9), so $p(x) = \prod_i p(x_i \mid x_1, \ldots, x_{i-1})$.

- **PixelRNN** (van den Oord et al., 2016): generate pixels from a corner; the dependency on previous pixels is an RNN (LSTM). Sequential generation is slow in both training and inference.
- **PixelCNN** (van den Oord et al., 2016): the dependency is a CNN over the context region (**masked convolution**). Training is faster (context values are known from training images, so convolutions run in parallel); generation is still slow: a 32×32 image needs 1024 forward passes.

Pros: explicit likelihood $p(x)$, easy to optimize, good samples. Con: sequential generation is slow. Improvements (PixelCNN++, Salimans et al., 2017): gated convolutions, shortcuts, discretized logistic loss, multi-scale, training tricks.

4 Latent Variable Models

Motivation. Only a small portion of the data space holds valid data, so a much more compact representation should exist.

Data are fixed-dimensional vectors $x \in \mathbb{R}^d$: $d = \text{width} \times \text{height} \times 3$ for images, $d = (\text{sentence length}) \times (\#\text{words})$ for sentences. A random point of $\mathbb{R}^d$ is almost never valid data. Hence a compact representation $z \in \mathbb{R}^m$ with $d \gg m$:

- **observable variable** x: observed directly from data;
- **latent variable** z: hidden, to be inferred from data.

Learning a representation ($x \to z \to$ output):

- Supervised (paired data (x, y)): compare the prediction for x with the known output,

$$\arg\min_{\psi,\phi} \sum_{(x,y)\in D} \mathcal{L}(p_\psi(g_\phi(x)), y).$$

- Unsupervised (data x only): model $p(x)$ by maximum likelihood (Chapter 2),

$$\arg\max_{\theta\in\Theta} \sum_i \log p_\theta(x^{(i)}).$$

Definition 11.1: latent variable model

A latent variable z (compact representation) generates the observation x (original representation), $p(x, z) = p_\theta(x \mid z)\, p(z)$:

1. sample $z \sim p(z)$ from a simple distribution;
2. generate $x \sim p_\theta(x \mid z)$, modeled by a neural network.

Why latent variables. They make data easy to interpret and manipulate.

- Attribute vector: $z =$ (angle, age, gender, color, ..., hair); sample z, generate a face conditioned on it.
- **Latent interpolation**: $z = \alpha z_1 + (1 - \alpha) z_2$ morphs one face into another.
- Changing one generative factor (e.g. hairstyle) changes only that attribute of the image.

Remaining questions. How to learn the model with no supervision on z? How to sample z (its distribution is unknown)? How to design the generator?

5 Autoencoders

Motivation. Learn an encoder mapping data to a compact z, much smaller than x yet keeping its most useful information, by reconstruction.

Definition 11.2: autoencoder

Encoder $q_\phi(z \mid x)$ maps x to the feature z; decoder $p_\theta(x \mid z)$ maps z back to the reconstruction $\hat{x}$. Both are trained on the reconstruction loss

$$\arg\min_{\theta,\phi} \|x - \hat{x}\|_2^2.$$

Ideally it learns the identity, yet the representation is useful: x must be squashed into a small z that still suffices for reconstruction. Learning is purely unsupervised. Example: 4-layer conv encoder, 4-layer upconv decoder.

Autoencoder for generation. Discard the encoder, sample a feature z, decode. Issue: the prior is unknown, and encoded points scatter irregularly in z-space. Fix: force the encoder distribution toward a simple prior,

$$\min_{\phi} D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z)), \qquad p(z) = \mathcal{N}(0, I),$$

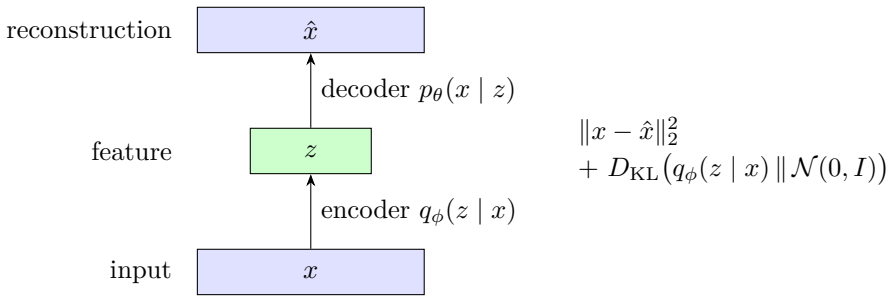
so $q_\phi(z \mid x) \approx \mathcal{N}(0, I)$ and $z \sim \mathcal{N}(0, I)$ can be sampled.

Definition 11.3: autoencoder as a generative model

Training:

$$\arg\min_{\theta,\phi} \sum_i \|x^{(i)} - \hat{x}^{(i)}\|_2^2 + D_{\mathrm{KL}}(q_\phi(z \mid x^{(i)}) \,\|\, p(z)).$$

Inference: sample $z^{(i)} \sim \mathcal{N}(0, I)$, then generate $\hat{x}^{(i)} \sim p_\theta(x \mid z^{(i)})$.



Autoencoder; the KL term makes sampling $z \sim \mathcal{N}(0, I)$ a valid way to generate.

What this objective does exactly is derived next from maximum likelihood.

6 Variational Autoencoder

Maximize $\sum_i \log p_\theta(x^{(i)})$ with the latent variable model $z \sim p(z)$, $x \sim p_\theta(x \mid z)$. For $x = x^{(i)}$,

$$\log p(x) = \log \int p(x, z)\, dz = \log \int p_\theta(x \mid z)\, p(z)\, dz.$$

Theorem 11.4: variational lower bound

$$\log p(x^{(i)}) \;\geq\; \mathbb{E}_{q_\phi(z|x^{(i)})}\big[\log p_\theta(x^{(i)} \mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x^{(i)}) \,\|\, p(z)\big).$$

The left side is the likelihood to maximize; the right side is the **variational lower bound**, so maximizing it maximizes the likelihood indirectly. The objective of the latent variable generative model:

$$\arg\max_{\theta,\phi} \; \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}_{q_\phi(z|x^{(i)})}\big[\log p_\theta(x^{(i)} \mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z \mid x^{(i)}) \,\|\, p(z)\big).$$